ОТЧЕТ ПО ИНДИВИДУАЛЬНОМУ ЗАДАНИЮ №3

По предмету “Архитектура компьютера и операционные системы”

ВАРИАНТ 48
ГОРОДСКАЯ СИСТЕМА ВОДОСНАБЖЕНИЯ

ВЫПОЛНЕНО:
Студенткой группы БПИ 253
Ким Алиной Алексеевной

АННОТАЦИЯ

Документ содержит отчёт о выполнении индивидуального задания №3 по дисциплине «Архитектура компьютера и операционные системы». Задача реализована как многопроцессное приложение для ОС Linux на языке программирования C. Взаимодействие процессов организовано с использованием разделяемой памяти POSIX (shm_open, mmap) и семафоров POSIX для синхронизации.

В отчёте приведены условие задачи, описание программных конструкций, отображающих сущности предметной области (насосы, резервуары, трубопроводы, районы-потребители, датчики, диспетчер), порядок выполнения алгоритма симуляции, описание пользовательского интерфейса и вариантов запуска программы. Особое внимание уделено разделению фактического состояния системы и представления диспетчера о нём, что моделирует работу в условиях ненадежной сети передачи данных. Реализованы дополнительные возможности: чтение конфигурации из файла, регулируемая задержка отображения, итоговая статистика, обработка сигнала прерывания пользователем (SIGINT) с корректным освобождением ресурсов, а также две стратегии управления насосами, переключаемые через указатели на функции. Реализованы оба варианта семафоров POSIX: именованные и неименованные (переключаемые макросом компиляции UNNAMED).

СОДЕРЖАНИЕ

АННОТАЦИЯ.....2
СОДЕРЖАНИЕ.....3
УСЛОВИЕ ЗАДАЧИ.....5
Входные параметры:.....5
Отображаемые события:.....5
Инварианты:.....6
Завершение:.....6
СУЩНОСТИ.....7
Резервуар (Reservoir):.....7
Насос (Pump):.....7
Трубопровод (Pipe):.....8
Район-потребитель (District):.....8
Точка графика потребления (SchedulePoint):.....8
Датчик (Sensor):.....9
Сообщения (Message):.....9
Диспетчер (Dispatcher):.....10
Система (System):.....10
АЛГОРИТМ ВЫПОЛНЕНИЯ.....12
КАК ЗАПУСКАТЬ.....14
Сборка:.....14
Запуск:.....14
Примеры:.....14
Варианты завершения:.....14
ОПИСАНИЕ ФОРМАТА CONFIG.TXT.....15
Общие правила:.....15
Пример:.....15
Описание параметров:.....16
ОПИСАНИЕ СТРАТЕГИЙ.....18
Жадная стратегия:.....18
Последовательная стратегия:.....18
РЕАЛИЗАЦИЯ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА.....19
Консольный вывод:.....19
Отображаемые события:.....19
Регулируемая задержка:.....19
Логирование в файл:.....19

УСЛОВИЕ ЗАДАЧИ

Городская система состоит из насосов, резервуаров, трубопроводов и районов-потребителей. Потребление воды изменяется по времени. Датчики периодически измеряют уровни резервуаров и передают показания диспетчеру; сообщения могут задерживаться, дублироваться или теряться.

Диспетчер включает и выключает насосы, перераспределяет потоки и поддерживает уровни в допустимых пределах. Каждый насос имеет производительность и потребляемую мощность, трубопровод - пропускную способность, а резервуар - минимальный и максимальный объём.

Общая доступная электрическая мощность ограничена, что при определенных значениях не позволяет включать одновременно все насосы.

Решения принимаются по последним полученным показаниям, поэтому модель должна различать фактическое состояние и представление диспетчера о нём.

Требуется разработать консольное приложение, моделирующее систему водоснабжения.

Входные параметры:

- схема соединений;
- объемы и начальные уровни резервуаров;
- число, производительность и мощность насосов;
- пропускная способность труб;
- графики потребления районов;
- пороговые уровни;
- задержка и вероятность потери показаний;
- стратегия управления и предел энергопотребления

Отображаемые события:

Необходимо отображать фактические уровни, измерение и передачу показаний, потерю и получение сообщения, включение насоса, изменение потоков, потребление района, достижение порога, переполнение и дефицит.

Инварианты:

- фактический объём резервуара не выходит за физический диапазон без регистрации аварии;
- поток по трубе не превышает ее пропускную способность;
- суммарная мощность насосов не превышает предел;
- сообщение датчика имеет время измерения и уникальный номер;
- повторное сообщение не обрабатывается как новое измерение.

Завершение:

Программа завершается после заданного периода моделирования либо при аварии, делающей дальнейшее снабжение невозможным, либо по прерыванию пользователем, если задан режим неограниченного по времени выполнения.

Диспетчер, датчики, насосы, резервуары и районы являются самостоятельными участниками модели.

СУЩНОСТИ

Резервуар (Reservoir):

Хранит физическое состояние ёмкости для воды. Резервуар не знает, откуда поступает вода и куда она уходит – эти связи описываются трубопроводами.

Поля:

- *id* - Уникальный идентификатор резервуара в системе. Используется для обращения к нему из датчиков, труб и диспетчера.
- *volume* - Фактический текущий объем воды в литрах. Изменяется функцией *water_physics()* на каждом тике. Это реальное состояние, которое видит только физика, но не диспетчер.
- *min_volume* - Минимально допустимый объем. При опускании ниже этого уровня срабатывает предупреждение, а диспетчер должен включить насос.
- *max_vilume* - Максимально допустимый объём (физический предел ёмкости). При превышении фиксируется авария переполнения. Диспетчер должен выключить насос при приближении к этому уровню.

Насос (Pump):

Описывает источник энергии для перекачки воды. Насос не знает, в какой резервуар он качает – эта связь хранится только в структуре *Pipe*, что реализует принцип единственного источника истины.

Поля:

- *id* – Уникальный идентификатор насоса. Используется диспетчером для управления (включение/выключение) и стратегией для принятия решений.
- *capacity* – Производительность насоса в литрах в секунду (л/с). Определяет, сколько воды насос может подать за один тик при включенном состоянии.
- *power* – Потребляемая электрическая мощность в ваттах (Вт). Используется стратегией для проверки ограничения по суммарной мощности диспетчера.

Трубопровод (Pipe):

Единственный источник истины о связях между объектами системы. Хранит информацию о том, откуда и куда течёт вода, а также ограничения потока. Именно через трубы стратегия узнаёт, какой резервуар обслуживает каждый насос.

Поля:

- *id* – Уникальный идентификатор трубопровода.
- *source_type* – Тип объекта-источника: *PUMP* (насос) или *RESERVOIR* (резервуар). Определяет, как интерпретировать поле *source_id*.
- *source_id* – Идентификатор объекта-источника. В сочетании с *source_type* однозначно указывает, откуда течёт вода.
- *destination_type* – Тип объекта-приемника: *RESERVOIR* (резервуар) или *DISTRICT* (район-потребитель).
- *destination_id* – Идентификатор объекта-приемника.
- *capacity* – Пропускная способность трубы в л/с. Функция *calculate_flow()* ограничивает реальный поток этим значением, реализуя инвариант ТЗ.

Район-потребитель (District):

Моделирует район города, потребляющий воду. Потребление изменяется во времени согласно графику. Район не знает, из какого резервуара он берёт воду – эта связь задается через трубопровод.

Поля:

- *id* – Уникальный идентификатор района.
- *schedule* – Динамический массив точек графика потребления. Каждая точка задает потребление на определённом временном интервале.
- *schedule_size* – Количество точек в графике потребления.
- *current_point* – Указатель на активную точку графика. Обновляется по мере продвижения модельного времени. Позволяет быстро получать текущее потребление без поиска по массиву.

Точка графика потребления (SchedulePoint):

Описывает один интервал в графике потребления района. График представляет собой кусочно-постоянную функцию, где каждая точка задает новый уровень потребления.

Поля:

- *time* – Время (в секундах модельного времени), начиная с которого действует данное потребление.
- *consumption* – Потребление воды в л/с на данном временном интервале. Действует до начала следующей точки графика.

Датчик (Sensor):

Периодически измеряет уровень воды в резервуаре и отправляет показания диспетчеру через модель сети. Датчик знает только о своём резервуаре и не взаимодействует с другими компонентами напрямую.

Поля:

- *id* – Уникальный идентификатор датчика. Используется в логах для отслеживания замеров.
- *reservoir_id* – Идентификатор резервуара, за которым наблюдает датчик.
- *measurement_frequency* – Интервал между измерениями в секундах. Определяет, как часто датчик "просыпается" для замера.
- *next_time* – Время следующего запланированного замера. Функция *sensors_work*() сравнивает *current_time* с этим полем и обновляет его после каждого замера.

Сообщения (Message):

Единица данных, передаваемая от датчика к диспетчеру через модель сети. Может задерживаться, дублироваться или теряться – это реализует ключевую особенность задачи: разделение фактического состояния и представления диспетчера.

Поля:

- *id* – Уникальный глобальный идентификатор сообщения. Генерируется счётчиком *global_messages_counter*. Используется кольцевым буфером для обнаружения дубликатов.
- *measurement_time* – Время, когда был сделан замер. Позволяет диспетчеру понимать, насколько "свежие" данные он получил.
- *reservoir_id* – Идентификатор резервуара, к которому относится замер.
- *level_value* – Значение уровня воды на момент замера. Именно это значение диспетчер записывает в свою "иллюзию" (*reservoirs_last_volume*).

- *delivery_time* – Время, когда сообщение будет доставлено диспетчеру. Устанавливается функцией *network_work()* с учетом случайной задержки. Если *delivery_time* <= *current_time*, сообщение считается доставленным.

Диспетчер (Dispatcher):

Автономный участник модели, принимающий решения на основе полученных (а не фактических) данных. Хранит "иллюзию" мира – свое представление о состоянии системы, которое может отличаться от реальности из-за задержек и потерь в сети.

Поля:

- *reservoirs_last_volume* – Динамический массив, хранящий последние известные диспетчеру уровни резервуаров. Обновляется только при получении сообщения от датчика. Именно на основе этих данных стратегия принимает решения.
- *on_pumps* – Динамический массив флагов состояния насосов (*true* – включен, *false* – выключен). Управляется стратегией и используется функцией water_physics() для расчета потоков.
- *max_power* – Максимально допустимая суммарная мощность всех включённых насосов. Стратегия проверяет этот лимит перед включением каждого насоса.
- *last_messages_ids* – Кольцевой буфер для хранения ID обработанных сообщений. Реализует защиту от повторной обработки дубликатов.
- *strategy* – Указатель на функцию стратегии управления. Позволяет переключаться между алгоритмами (например, *greedy_strategy* и *sequential_strategy*) без изменения кода диспетчера. Реализует принцип открытости/закрытости (Open/Closed Principle).
- strategy_type - тип стратегии (перечисление StrategyType), используется для инициализации указателя strategy в процессе диспетчера.

Система (System):

Контекстная структура, объединяющая все компоненты модели. Хранит как фактическое состояние, так и очередь сообщений, и диспетчера. Является «контейнером» для всех участников симуляции и размещается в разделяемой памяти POSIX.

Поля:

- current_time - текущее модельное время в секундах. Увеличивается на 1 в конце каждого тика главного цикла.
- emergency - флаг фатальной аварии. При установке в true главный цикл завершает симуляцию.
- stop - флаг корректного завершения (устанавливается при достижении max_time или по сигналу).
- init_done - флаг завершения инициализации разделяемой памяти (используется дочерними процессами для проверки готовности).
- clock_pid - PID главного процесса (Clock), используется для обнаружения его гибели.
- seed - значение для инициализации генератора случайных чисел (обеспечивает воспроизводимость).
- max_time - максимальное время симуляции.
- reservoirs - массив резервуаров.
- reservoir_number - количество резервуаров.
- pumps - массив насосов.
- pumps_number - количество насосов.
- districts - массив районов-потребителей.
- districts_number - количество районов.
- pipes - массив трубопроводов.
- pipes_number - количество трубопроводов.
- sensors - массив датчиков.
- sensors_number - количество датчиков.
- messages - массив - очередь сообщений «в полёте». Сообщения добавляются датчиками и удаляются диспетчером после обработки.
- message_count - текущее количество сообщений в очереди.
- message_capacity - выделенная емкость массива сообщений.
- global_messages_counter - глобальный счетчик для генерации уникальных ID сообщений.
- loss_probability - вероятность потери сообщения в сети (в процентах).
- delay_probability - вероятность задержки сообщения.
- max_delay - максимальная задержка сообщения в секундах.
- dublicate_probability - вероятность дублирования сообщения.
- dispatcher - вложенная структура диспетчера.
- unnamed[] - массив неименованных семафоров POSIX (используется при компиляции с макросом UNNAMED).

АРХИТЕКТУРА МНОГОПРОЦЕССНОЙ СИМУЛЯЦИИ

В отличие от первой версии программы, где все события обрабатывались последовательно в едином цикле, текущая версия реализует многопроцессную архитектуру на базе системного вызова fork(). Ключевым принципом проектирования стало отображение сущностей предметной области, обладающих поведением, в отдельные процессы выполнения, в то время как пассивные структуры данных размещены в разделяемой памяти с защитой доступа через семафоры POSIX.

Отображение сущностей в потоки:

Было принято решение создать отдельные процессы для каждой сущности, обладающей активным поведением:

- Clock Process (главный процесс) - управляет модельным временем (current_time), инициирует начало каждого такта симуляции через фазовые семафоры, координирует завершение работы всех дочерних процессов.
- Network Process - моделирует ненадежную среду: задерживает, дублирует или теряет сообщения.
- Dispatcher Process - получает сообщения от датчиков, обновляет "иллюзию" мира, принимает решения о включении/выключении насосов через стратегию.
- District Processes (N штук) - моделируют районы-потребители, обновляют графики потребления.
- Pipe Processes (N штук) - рассчитывают потоки воды через трубы, обновляют объемы резервуаров.
- Sensor Processes (N штук) - периодически снимают показания с резервуаров и формируют сообщения для диспетчера.

Резервуары и насосы являются пассивными сущностями: они не принимают решений и не инициируют действия, поэтому для них не создаются отдельные процессы. Их состояние хранится в разделяемой памяти и обновляется процессами труб.

В файле main.c главный процесс создает дочерние процессы с помощью системного вызова fork().

Каждый дочерний процесс вызывает функцию run_worker(), которая определяет свою роль и переходит в цикл ожидания семафоров.

Организация IPC и синхронизации:

Разделяемая память POSIX:

Все процессы работают с общей структурой System, которая размещается в разделяемой памяти POSIX. Для этого используются системные вызовы:

- shm_open() - создание или открытие именованного сегмента разделяемой памяти (/wss_shm).
- ftruncate() - установка размера сегмента равным sizeof(System).
- mmap() - отображение сегмента в адресное пространство процесса с флагом MAP_SHARED.

Главный процесс (Clock) создает разделяемую память в функции ipc_create(), а дочерние процессы подключаются к ней через ipc_attach().

Семафоры POSIX:

Для синхронизации процессов используются семафоры POSIX. В проекте реализованы оба варианта (переключаемые макросом компиляции UNNAMED):

Вариант 1: Именованные семафоры POSIX (по умолчанию)

- Создаются с помощью sem_open() с уникальными именами (/wss_sem_0, /wss_sem_1, ...).
- Доступны всем процессам по имени.
- Удаляются через sem_unlink() при завершении.

Вариант 2: Неименованные семафоры в разделяемой памяти (при компиляции с -DUNNAMED)

- Инициализируются с помощью sem_init() непосредственно в структуре System (поле unnamed[]).
- Размещаются в разделяемой памяти, поэтому доступны всем процессам.
- Уничтожаются через sem_destroy() при завершении.

Назначение семафоров:

В проекте используются следующие семафоры (определены в utils/sem_enum.h):

- SEM_MTX_RES - мьютекс для защиты доступа к резервуарам (используется процессами труб и датчиков).

- SEM_MTX_MSG - мьютекс для защиты очереди сообщений (используется процессами сети, диспетчера и датчиков).
- SEM_READY - семафор для сигнализации о готовности дочерних процессов (используется при старте).
- SEM_EXIT - семафор для сигнализации о завершении дочернего процесса.
- SEM_GO_NETWORK, SEM_GO_DISPATCHER, SEM_GO_DISTRICT[i], SEM_GO_PIPE[i], SEM_GO_SENSOR[i] - семафоры «старт фазы», используются главным процессом для запуска рабочих процессов на каждом такте.
- SEM_DONE_NETWORK, SEM_DONE_DISPATCHER, SEM_DONE_DISTRICT, SEM_DONE_PIPE, SEM_DONE_SENSOR - семафоры «завершение фазы», используются рабочими процессами для сигнализации о завершении работы на текущем такте.

Фазовая синхронизация:

Для обеспечения строгого порядка выполнения процессов на каждом такте используется фазовая синхронизация через барьеры на семафорах. Алгоритм одного такта:

1. Главный процесс (Clock) вызывает run_phase() для каждой фазы в строгом порядке:
   - **Фаза 1:** Network - процесс сети обрабатывает сообщения (потери, задержки, дубликаты).
   - **Фаза 2:** Dispatcher - процесс диспетчера получает сообщения и принимает решения.
   - **Фаза 3:** Districts - процессы районов обновляют графики потребления.
   - **Фаза 4:** Pipes - процессы труб рассчитывают потоки и обновляют объемы резервуаров.
   - **Фаза 5:** Sensors - процессы датчиков снимают показания и создают сообщения.
2. Функция run_phase(phase, go_base, n):
   - Освобождает n семафоров SEM_GO_*, запуская соответствующие рабочие процессы.

- Ожидает n семафоров SEM_DONE_*, дожидаясь завершения всех процессов текущей фазы.
- Использует sem_timedwait() с таймаутом для обнаружения зависаний.

3. Каждый рабочий процесс в цикле:
   - Ожидает свой семафор SEM_GO_* (с таймаутом).
   - Выполняет свою работу (вызывает соответствующую функцию: network_work(), dispatcher_work() и т.д.).
   - Освобождает семафор SEM_DONE_*, сигнализируя о завершении.

Такая архитектура гарантирует, что процессы выполняются строго в заданном порядке, несмотря на их асинхронную природу.

Защита от зависаний:

Для предотвращения зависаний при гибели главного процесса используется механизм проверки «живости» Clock-процесса:

1. В структуре System хранится clock_pid - PID главного процесса.
2. В функции acquire_ex() (файл ipc.c) при ожидании семафора периодически проверяется: kill(clock_pid, 0).
3. Если главный процесс погиб (вернул ESRCH), рабочий процесс устанавливает g_sys->stop = 1 и корректно завершается.

Это гарантирует, что при падении главного процесса все дочерние процессы не зависнут навсегда на семафорах, а корректно завершатся.

Обработка сигналов:

Для обработки сигнала SIGINT (Ctrl+C) используется функция init_signal(), которая устанавливает обработчик сигнала. Обработчик устанавливает глобальный флаг g_stop, который проверяется:

- Главным процессом в цикле симуляции.
- Рабочими процессами при ожидании семафоров (через acquire_ex()).

При установке флага g_stop все процессы корректно завершают работу, освобождая ресурсы.

Очистка ресурсов:

При завершении программы (нормальном или по сигналу) вызывается функция clean_system(), которая:

1. Уничтожает все семафоры (sem_unlink() для именованных или sem_destroy() для неименованных).
2. Отключает разделяемую память (munmap()).
3. Удаляет сегмент разделяемой памяти (shm_unlink()).

Это гарантирует, что в операционной системе не останется «висячих» семафоров и сегментов памяти.

Алгоритм выполнения:

1. network_work - обработка сообщений в сети

Процесс сети работает с очередью сообщений «в полёте»:

- Захватывает семафор SEM_MTX_MSG для защиты очереди.
- Для каждого сообщения «бросает кубики» (rand()):
  - С вероятностью loss_probability сообщение удаляется (потеряно).
  - С вероятностью delay_probability устанавливается delivery_time = current_time + random_delay.
  - С вероятностью dublicate_probability создаётся копия сообщения с тем же ID.
- Освобождает семафор SEM_MTX_MSG.

2. dispatcher_work - получение данных и принятие решений

Процесс диспетчера обрабатывает доставленные сообщения:

- Захватывает семафор SEM_MTX_MSG.
- Проходит по очереди сообщений.
- Если delivery_time <= current_time - сообщение доставлено:
  - Проверяет ID через кольцевой буфер last_messages_ids: если ID уже есть - это дубликат, игнорируем.
  - Иначе: обновляет reservoirs_last_volume[reservoir_id] = message.volume.
  - Добавляет ID в кольцевой буфер.
  - Удаляет сообщение из очереди.
- Освобождает семафор SEM_MTX_MSG.
- Вызывает sys->dispatcher.strategy() для принятия решений о включении/выключении насосов.

3. district_work - обновление потребления районов

Каждый процесс района обновляет свой график потребления:

- Проходит по точкам графика.

- Находит активную точку для текущего времени (обновляет current_point).
- Логирует текущее потребление.

4. water_physics - расчет потоков воды

Каждый процесс трубы рассчитывает поток:

- Захватывает семафор SEM_MTX_RES для защиты резервуаров.
- Вычисляет реальный поток через calculate_flow(pipe, sys):
    - Поток ограничен: min(желание_источника, пропускная_способность_трубы).
    - Если источник - насос, поток равен capacity при включенном насосе.
    - Если источник - резервуар, поток ограничен его текущим объемом и потреблением района.
- Обновляет объемы резервуаров: volume += inflow - outflow.
- Освобождает семафор SEM_MTX_RES.

5. sensors_work - измерение уровней

Каждый процесс датчика периодически снимает показания:

- Проверяет: current_time >= next_time.
- Если да:
    - Захватывает семафор SEM_MTX_RES.
    - Считывает реальный уровень резервуара.
    - Освобождает семафор SEM_MTX_RES.
    - Захватывает семафор SEM_MTX_MSG.
    - Создает сообщение с измеренным уровнем, добавляет в очередь.
    - Освобождает семафор SEM_MTX_MSG.
    - Обновляет next_time += measurement_frequency.

6. check_emergency - проверка аварий

Главный процесс после каждого такта проверяет:

- Каждый резервуар:
    - volume <= 0 - фатальная авария (высыхание).
    - volume >= max_volume - фатальная авария (переполнение).
    - volume < min_volume - предупреждение (не останавливает симуляцию).
- Суммарную мощность включенных насосов:
    - Если total_power > max_power - фатальная авария (превышение лимита).

КАК ЗАПУСКАТЬ

Сборка:

make

Запуск:

./water_supply <config_file> [delay_ms] [max_time] [seed]

- *config_file - путь к конфигурационному файлу (обязательный).*
- *delay_ms - задержка между тиками в миллисекундах (необязательный, по умолчанию 500).*
- *max_time - максимальное время симуляции в тиках (необязательный, по умолчанию 100).*
- *seed - значение для инициализации генератора случайных чисел (необязательный, по умолчанию 1). Обеспечивает воспроизводимость поведения сети.*

Примеры:

```
# Базовый запуск с задержкой 500 мс
./water_supply config.txt

# Быстрая симуляция (без задержки) с максимальным временем 200 тиков и seed 42
./water_supply config.txt 0 200 42

# Медленная симуляция для демонстрации (1 секунда на тик)
./water_supply config.txt 1000
```

Варианты завершения:

- *Завершение по времени:* достигнут MAX_TIME (например, 100 секунд);
- *Завершение по аварии:* объём резервуара ≤ 0 (высыхание) или ≥ max_volume (переполнение);
- *Завершение по прерыванию (Ctrl+C):* пользователь нажал Ctrl+C во время работы. Обработчик сигнала устанавливает флаг g_stop, все процессы корректно завершаются, выводится статистика, освобождаются ресурсы IPC.

ОПИСАНИЕ ФОРМАТА CONFIG.TXT

Общие правила:

- Каждая строка – либо комментарий (начинается с #), либо директива
- Директива состоит из ключевого слова и параметров, разделенных пробелами
- Пустые строки игнорируются

Пример:

```
# ============================================================
# Конфигурация системы водоснабжения
# ============================================================

# Резервуары
# Формат: reservoir <id> <min_volume> <max_volume> <initial_volume>
reservoirs 2
reservoir 0 20.0 100.0 60.0
reservoir 1 30.0 150.0 90.0

# Насосы
# Формат: pump <id> <capacity> <power>
pumps 2
pump 0 10.0 50.0
pump 1 15.0 80.0

# Районы с графиками потребления
# Формат: district <id> <schedule_size> <time1> <consumption1> <time2> <consumption2> ...
districts 2
district 0 3 0 8.0 10 12.0 20 6.0
district 1 2 0 12.0 15 8.0

# Трубопроводы
# Формат: pipe <id> <source_type> <source_id> <dest_type> <dest_id> <capacity>
# source_type/dest_type: pump | reservoir | district
pipes 4
```

pipe 0 pump 0 reservoir 0 20.0
pipe 1 reservoir 0 district 0 15.0
pipe 2 pump 1 reservoir 1 25.0
pipe 3 reservoir 1 district 1 20.0

Датчики
Формат: sensor <id> <reservoir_id> <measurement_frequency>
sensors 2
sensor 0 0 3
sensor 1 1 3

Параметры сети
Формат: network <loss_probability> <max_delay> <duplicate_probability>
Вероятности в процентах (0-100)
network 15.0 2 10.0

Диспетчер
Формат: dispatcher <max_power> <strategy_type>
strategy_type: greedy | sequential
dispatcher 130 greedy

Описание параметров:

- reservoirs N - Количество резервуаров - reservoirs 2
- reservoir id min max init - Резервуар: ID, мин/макс объем, начальный - reservoir 0 20.0 100.0 60.0
- pumps N - Количество насосов - pumps 2
- pump id capacity power - Насос: ID, производительность, мощность - pump 0 10.0 50.0
- districts N - Количество районов - districts 2
- district id size [t c ...] - Район: ID, размер графика, пары (время, потребление) - district 0 2 0 8.0 10 12.0
- pipes N - Количество труб - pipes 4
- pipe id src_type src_id dst_type dst_id cap - Труба: ID, типы/ID источников и приемников, пропускная способность - pipe 0 pump 0 reservoir 0 20.0
- sensors N - Количество датчиков - sensors 2

- sensor id reservoir_id freq - Датчик: ID, ID резервуара, частота замеров - sensor 0 0 3
- network loss% delay dup% - Параметры сети: вероятность потери, макс задержка, вероятность дублирования - network 15.0 2 10.0
- dispatcher max_power strategy - Диспетчер: лимит мощности, тип стратегии

ОПИСАНИЕ СТРАТЕГИЙ

Жадная стратегия:

Принцип: На каждом шаге выбирает самую критическую проблему и решает её первой.

Алгоритм:

1. Для каждого насоса находит целевой резервуар (через Pipe)
2. Считает критичность резервуара:
3. Сортирует насосы по критичности (от самой отрицательной к положительной)
4. Жадно включает насосы, начиная с самых критичных, пока не закончится мощность
5. Выключает насосы, если уровень достиг целевого значения (⅔ от максимального + минимального значений)

Последовательная стратегия:

Принцип: Включает насосы в порядке индексов, пока хватает мощности.

Алгоритм:

1. Проходит по насосам от 0 до N-1
2. Если уровень резервуара ниже минимума – пытается включить насос
3. Если не хватает мощности – пропускает
4. Выключает насосы при достижении целевого уровня

РЕАЛИЗАЦИЯ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

Консольный вывод:

- Все события выводятся в стандартный поток stdout с префиксом времени;
- Формат времени: [T=XXX] – три цифры с ведущими нулями (например, [T=007]);

Отображаемые события:

- Фактические уровни
- Измерение показаний
- Передача показаний
- Потеря сообщения
- Получение сообщения
- Включение насоса
- Выключение насоса
- Изменение потоков
- Потребление района
- Достижение порога
- Переполнение
- Дефицит
- Дубликаты
- Системные сообщения о запуске/завершении процессов

Регулируемая задержка:

- Для наглядности используется функция msleep(), реализованная через системный вызов nanosleep().

Логирование в файл:

- Все события дублируются в файл лога через функцию log_event(), использующую системный вызов write() для атомарной записи.